Gap-Fill Master Cheatsheet

Covers: Dynamic Programming, Backtracking, Sliding Window, Greedy, Bit Manipulation, LLD/OOP Class Design. These were the confirmed missing pieces after everything else built so far.

PART 1 — DYNAMIC PROGRAMMING

Core idea: break a problem into overlapping subproblems, store results so you never recompute. Two styles: Top-Down (recursion + memo array) and Bottom-Up (iterative table).

1.1 Fibonacci (the simplest DP shape — learn this pattern first)

```
// Top-down with memoization
int memo[100];
int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}

// Bottom-up (preferred in interviews - no recursion overhead)
int fibDP(int n) {
    if (n <= 1) return n;
    vector<int> dp(n+1);
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

1.2 Climbing Stairs (1 or 2 steps at a time, count ways)

Trigger words: 'count ways to reach', 'number of distinct ways' -> this exact Fibonacci shape.

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

1.3 0/1 Knapsack (the most reused DP shape in interviews)

Trigger words: 'maximize value with weight limit', 'select items without exceeding capacity'.

```
int knapsack(vector<int>& wt, vector<int>& val, int n, int W) {
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}
```

1.4 Longest Common Subsequence (LCS)

Trigger words: 'common subsequence', 'edit distance family', comparing two strings/sequences.

```
int lcs(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i-1] == b[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[n][m];
}
```

1.5 Coin Change (minimum coins to make amount)

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount+1, INT_MAX);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int c : coins) {
            if (c <= i && dp[i-c] != INT_MAX)
                dp[i] = min(dp[i], dp[i-c] + 1);
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
```

1.6 House Robber (no two adjacent picks)

Trigger words: 'cannot pick adjacent elements', maximize sum with a gap constraint.

```
int rob(vector<int>& nums) {
    int prev = 0, curr = 0;
    for (int n : nums) {
        int temp = max(curr, prev + n);
        prev = curr;
        curr = temp;
    }
    return curr;
}
```

DP Trigger Word -> Pattern Map

Trigger phrase	DP Pattern
'maximize value within capacity/weight'	0/1 Knapsack
'longest common ...' between two strings	LCS
'minimum coins/steps to reach X'	Coin Change / BFS-DP
'cannot pick adjacent'	House Robber pattern
'count distinct ways to reach N'	Fibonacci-shaped DP
'longest increasing subsequence'	LIS - O(n^2) or O(n log n) with binary search

PART 2 — BACKTRACKING

Core idea: try a choice, recurse, if it fails or you've explored it fully, UNDO the choice (backtrack) and try the next one. Always has a 'choose -> explore -> un-choose' shape.

2.1 The Universal Backtracking Template

```
void backtrack(/* state */) {
    if (/* base case - found a solution */) {
        result.push_back(current);   // record it
        return;
    }
    for (/* each choice available */) {
        // 1. CHOOSE
        current.push_back(choice);
        // 2. EXPLORE
        backtrack(/* updated state */);
        // 3. UN-CHOOSE (backtrack)
        current.pop_back();
    }
}
```

2.2 Subsets (all possible subsets of a set)

```
vector<vector<int>> result;
vector<int> current;
void backtrack(vector<int>& nums, int start) {
    result.push_back(current);
    for (int i = start; i < nums.size(); i++) {
```

```
        current.push_back(nums[i]);
        backtrack(nums, i+1);
        current.pop_back();
    }
}
```

2.3 Permutations (all orderings)

```
void permute(vector<int>& nums, vector<bool>& used, vector<int>& current, vector<vector<int>>& result) {
    if (current.size() == nums.size()) { result.push_back(current); return; }
    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) continue;
        used[i] = true;
        current.push_back(nums[i]);
        permute(nums, used, current, result);
        current.pop_back();
        used[i] = false;
    }
}
```

2.4 N-Queens (classic constraint-satisfaction backtracking)

```
bool isSafe(vector<string>& board, int row, int col, int n) {
    for (int i = 0; i < row; i++) if (board[i][col] == 'Q') return false;
    for (int i=row-1, j=col-1; i>=0 && j>=0; i--, j--) if (board[i][j]=='Q') return false;
    for (int i=row-1, j=col+1; i>=0 && j<n; i--, j++) if (board[i][j]=='Q') return false;
    return true;
}
void solve(vector<string>& board, int row, int n, vector<vector<string>>& result) {
    if (row == n) { result.push_back(board); return; }
    for (int col = 0; col < n; col++) {
        if (isSafe(board, row, col, n)) {
            board[row][col] = 'Q';
            solve(board, row+1, n, result);
            board[row][col] = '.';   // backtrack
        }
    }
}
```

2.5 Combination Sum (numbers that add up to target, reuse allowed)

```
void combSum(vector<int>& candidates, int target, int start, vector<int>& current, vector<vector<int>>& result) {
    if (target == 0) { result.push_back(current); return; }
    if (target < 0) return;
    for (int i = start; i < candidates.size(); i++) {
        current.push_back(candidates[i]);
        combSum(candidates, target - candidates[i], i, current, result);  // i, not i+1 -> reuse allowed
        current.pop_back();
    }
}
```

PART 3 — SLIDING WINDOW (dedicated, with code)

Core idea: maintain a window [left, right] over an array/string, expand right, shrink left when a condition breaks. Avoids O(n^2) brute force.

3.1 Fixed-Size Window (max sum of subarray size k)

```
int maxSumFixed(vector<int>& nums, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];
    int maxSum = windowSum;
    for (int i = k; i < nums.size(); i++) {
        windowSum += nums[i] - nums[i-k];   // slide: add new, remove old
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}
```

3.2 Variable-Size Window (smallest subarray with sum >= target)

```
int minSubArrayLen(int target, vector<int>& nums) {
    int left = 0, sum = 0, minLen = INT_MAX;
    for (int right = 0; right < nums.size(); right++) {
```

```
        sum += nums[right];
        while (sum >= target) {
            minLen = min(minLen, right - left + 1);
            sum -= nums[left];
            left++;          // shrink window
        }
    }
    return minLen == INT_MAX ? 0 : minLen;
}
```

3.3 Longest Substring Without Repeating Characters

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char,int> lastSeen;
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.size(); right++) {
        if (lastSeen.count(s[right]) && lastSeen[s[right]] >= left)
            left = lastSeen[s[right]] + 1;     // jump window start past duplicate
        lastSeen[s[right]] = right;
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

PART 4 — GREEDY ALGORITHMS

Core idea: at each step, make the locally optimal choice and never look back. Works only when the problem has 'greedy choice property' (local optimum leads to global optimum) — say this caveat out loud in interviews.

4.1 Activity Selection / Interval Scheduling

Trigger words: 'maximum number of non-overlapping meetings/intervals'.

```
int maxActivities(vector<pair<int,int>>& intervals) {
    sort(intervals.begin(), intervals.end(),
         [](auto& a, auto& b) { return a.second < b.second; });   // sort by END time
    int count = 1, lastEnd = intervals[0].second;
    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i].first >= lastEnd) {
            count++;
            lastEnd = intervals[i].second;
        }
    }
    return count;
}
```

4.2 Jump Game (can you reach the last index?)

```
bool canJump(vector<int>& nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (i > maxReach) return false;
        maxReach = max(maxReach, i + nums[i]);
    }
    return true;
}
```

4.3 Gas Station (circular route, find starting point)

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.size(); i++) {
        int diff = gas[i] - cost[i];
        total += diff;
        tank += diff;
        if (tank < 0) { start = i + 1; tank = 0; }   // reset start point
    }
    return total >= 0 ? start : -1;
}
```

PART 5 — BIT MANIPULATION (deeper)

5.1 Core operators cheat table

Operation	Meaning
n & 1	Check if n is odd (last bit)
n >> 1	Divide by 2
n << 1	Multiply by 2
n & (n-1)	Clears the lowest set bit
n & (-n)	Isolates the lowest set bit
a ^ a = 0	XOR with self cancels out
a ^ 0 = a	XOR with 0 is identity

5.2 Single Number (every element appears twice except one)

Idea: XOR all elements. Pairs cancel to 0, leaving only the unique one.

```
int singleNumber(vector<int>& nums) {
    int result = 0;
    for (int n : nums) result ^= n;
    return result;
}
```

5.3 Count Set Bits (number of 1s in binary)

```
int countBits(int n) {
    int count = 0;
    while (n) {
        n = n & (n - 1);   // clears lowest set bit each time
        count++;
    }
    return count;
}
```

5.4 Check Power of Two

```
bool isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;   // power of 2 has exactly one set bit
}
```

5.5 Find Missing Number (0 to n, one missing)

```
int missingNumber(vector<int>& nums) {
    int result = nums.size();
    for (int i = 0; i < nums.size(); i++)
        result ^= i ^ nums[i];
    return result;
}
```

PART 6 — LLD / OOP CLASS DESIGN

Different from the DSA versions of the same problems. Here you design CLASSES with state and behavior, not just a processing function. Interviewers check: encapsulation, single responsibility, extensibility (open/closed principle).

6.1 Elevator System (the lift up/down problem, OOP version)

```
enum class Direction { UP, DOWN, IDLE };

class Request {
public:
    int floor;
    Direction direction;
    Request(int f, Direction d) : floor(f), direction(d) {}
};

class Elevator {
    int currentFloor;
```

```
    Direction state;
    priority_queue<int> upRequests;                          // max-heap, closest above first
    priority_queue<int, vector<int>, greater<int>> downRequests;  // min-heap, closest below first

public:
    Elevator() : currentFloor(0), state(Direction::IDLE) {}

    void addRequest(Request req) {
        if (req.floor > currentFloor) upRequests.push(req.floor);
        else if (req.floor < currentFloor) downRequests.push(req.floor);
    }

    void moveStep() {
        if (state == Direction::UP || (!upRequests.empty() && state == Direction::IDLE)) {
            if (!upRequests.empty()) {
                currentFloor = upRequests.top(); upRequests.pop();
                state = upRequests.empty() ? Direction::IDLE : Direction::UP;
            }
        } else if (!downRequests.empty()) {
            currentFloor = downRequests.top(); downRequests.pop();
            state = downRequests.empty() ? Direction::IDLE : Direction::DOWN;
        }
    }
};
```

Key design point to say out loud: 'I separate up-requests and down-requests into two heaps so the elevator always serves the closest floor in its current direction before reversing — this is the SCAN/elevator disk-scheduling algorithm.'

6.2 Parking Lot System

```
enum class VehicleType { MOTORCYCLE, CAR, TRUCK };

class Vehicle {
public:
    string plate;
    VehicleType type;
    Vehicle(string p, VehicleType t) : plate(p), type(t) {}
};

class ParkingSpot {
public:
    int spotNumber;
    VehicleType allowedType;
    bool isOccupied;
    Vehicle* parkedVehicle;

    ParkingSpot(int num, VehicleType type) : spotNumber(num), allowedType(type), isOccupied(false), parkedVehicle(nullptr) {}

    bool park(Vehicle* v) {
        if (isOccupied || v->type != allowedType) return false;
        parkedVehicle = v;
        isOccupied = true;
        return true;
    }
    void leave() { parkedVehicle = nullptr; isOccupied = false; }
};

class ParkingLot {
    vector<ParkingSpot> spots;
public:
    ParkingLot(int numSpots, VehicleType defaultType) {
        for (int i = 0; i < numSpots; i++) spots.push_back(ParkingSpot(i, defaultType));
    }
    ParkingSpot* findAvailableSpot(VehicleType type) {
        for (auto& spot : spots)
            if (!spot.isOccupied && spot.allowedType == type) return &spot;
        return nullptr;
    }
};
```

Key design point: each VehicleType maps to spot compatibility -> this is naturally extensible (add ELECTRIC type with charging spots later without breaking existing code = Open/Closed Principle).

6.3 Library Management System (inheritance-focused)

```
class Item {
public:
    string title;
```

```
    bool isAvailable;
    Item(string t) : title(t), isAvailable(true) {}
    virtual int loanDurationDays() = 0;   // pure virtual - forces subclasses to define
};

class Book : public Item {
public:
    Book(string t) : Item(t) {}
    int loanDurationDays() override { return 14; }
};

class DVD : public Item {
public:
    DVD(string t) : Item(t) {}
    int loanDurationDays() override { return 7; }
};

class Library {
    vector<Item*> items;
public:
    void addItem(Item* item) { items.push_back(item); }
    bool checkout(string title) {
        for (auto* item : items) {
            if (item->title == title && item->isAvailable) {
                item->isAvailable = false;
                return true;
            }
        }
        return false;
    }
};
```

Key design point: abstract Item base class with pure virtual loanDurationDays() forces every new item type (Magazine, Equipment) to define its own rule -> demonstrates polymorphism + abstraction together.

6.4 Vending Machine (state machine design)

```
enum class VMState { IDLE, HAS_MONEY, DISPENSING };

class VendingMachine {
    VMState state;
    int insertedAmount;
    map<string,int> itemPrices = {{"Coke", 100}, {"Chips", 150}};
public:
    VendingMachine() : state(VMState::IDLE), insertedAmount(0) {}

    void insertCoin(int amount) {
        insertedAmount += amount;
        state = VMState::HAS_MONEY;
    }

    bool selectItem(string item) {
        if (state != VMState::HAS_MONEY) return false;
        if (itemPrices[item] > insertedAmount) return false;
        state = VMState::DISPENSING;
        insertedAmount -= itemPrices[item];
        state = VMState::IDLE;
        return true;
    }
};
```

Key design point: explicit state enum makes illegal transitions impossible (can't dispense without money) -> this is the core idea interviewers want to see articulated, not just working code.

Priority To-Do (in order)

1. Code Subsets + Permutations from memory (Part 2) - establishes the backtracking template in muscle memory
2. Code Knapsack + LCS from memory (Part 1) - the two most reused DP shapes
3. Code the 3 Sliding Window problems blind (Part 3)
4. Memorize the bit trick table (Part 5) - quick wins, low effort

5. Be ready to verbally walk through Elevator + Parking Lot class design (Part 6) without full code - structure matters more than syntax here

6. Greedy - lowest priority of the 6, but know Activity Selection cold since it repeats often